certora

Security Assessment & Formal Verification Final Report

Solana Stake Pool

September 2025

certora

Table of Contents

certora

Project Summary

Project Scope

Project Name	Repository (link)	Latest Commit Hash	Platform
Stake Pool	https://github.com/solana-program/stake	22834f8	Solana

Project Overview

This document describes the specification and verification of **Stake Pool** using the Certora Prover. The work was undertaken from **June 26, 2025** to **Sept. 15, 2025**.

The following contract list is included in our scope:

```
{
      program/src/*
}
```

The Certora Prover demonstrated that the implementation of the **Solana** contracts above is correct with respect to the formal rules written by the Certora team. During the verification process, the Certora team discovered bugs in the Solana contracts code, as listed on the following page. For each bug, relevant issues from the accompanying security audit report are mentioned when possible.

Protocol Overview

Solana Stake Pool enables the creation of pools where users can deposit their SOL, which is then staked and managed by the pool's administrators.
The revenue from staking would be distributed between the depositors, while some of it goes as a fee to the manager.
The users' funds are guaranteed to be safe, even from the manager and staker roles, as they can only manage the stake or set the fees, but don’t have direct access to the deposited SOL.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	4	4	4
Medium	4	4	4
Low	5	5	3
Informational	2	2	-
Total	15		

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
H-01	An attacker can permanently freeze funds in the pool by hijacking a transient account	High	Fixed
H-02	A hijacked account can be used to steal funds from the pool, by removing and re-adding the validator	High	Fixed
H-03	WithdrawStake and removal after cluster-restart don't reset preferred validators after their removal, DoS-ing withdrawals and deposits	High	Fixed
H-04	WithdrawStake allows draining (and hijacking) the reserve stake account, leading to loss of funds.	High	Fixed
M-01	The pool keeps accounting for a hijackable stake account after cluster restart.	Medium	Fixed
M-02	WithdrawStake can remove a validator with transient stake, allowing manipulation of share price	Medium	Partly Fixed
M-03	Non-active validators block WithdrawStake despite not allowing withdrawing from them	Medium	Fixed

M-04	UpdateValidatorListBalances misses accounting for validator stake funds.	Medium	Fixed
L-01	Staker can call DecreaseStake on funds when the validator's status is DeactivatingValidator, leading to unexpected funds in the transient account.	Low	Fixed
L-02	WithdrawStake can be DoS-ed with a huge validators list	Low	Fixed
L-03	RemoveValidatorFromPool transitions status to DeactivatingValidator even when there could be funds in a transient stake account.	Low	Fixed
L-04	WithdrawStake and DecreaseValidatorStake mix physical and virtual accounting leading to an inconsistent status.	Low	Not yet fixed
L-05	Intermediate computation in get_lamports_per_pool_token could overflow.	Low	Not yet fixed

High Severity Issues

H-01 An attacker can permanently freeze funds in the pool by hijacking a transient account

Severity: **High**	Impact: **High**	Likelihood: **Medium**
Files: program/src/processor.rs	Status: Fixed	Violated Properties: stake_usable_update_validator_list_balance_no_merge

Description: Transient accounts created by the pool can be hijacked when they're merged into the reserve or the main stake account (this can be done by re-initializing a merged account, since after merge the stake state changes to `Uninitalized` and the stake program allows anybody to initialize an uninitialized account).
The pool is aware of that and doesn't account for accounts that aren't usable by the pool.
But this check alone isn't sufficient, as an attacker can make the account usable by the pool but delegate it to another validator. This can DoS some operations of the pool, since it'd attempt to merge from/to the hijacked account and it'd fail since it's delegated to a different validator.

Exploit Scenario #1:

- Bob monitors the pool and hijacks every transient account when it's merged to the pool
  - Hijacking is done by re-initializing a merged account, since after merge the stake state changes to `Uninitalized` and the stake program allows anybody to initialize an uninitialized account
  - Bob delegates the stake to another validator
- The staker calls `RemoveValidatorFromPool`, changing the status to `DeactivatingValidator`
- Bob makes the account usable by the pool

- Now, on every call to `UpdateValidatorListBalance` (when `no_merge=false`) the program would attempt to merge the transient account to the main stake account, but that would fail because it's deactivating and delegated to a different validator
    - This would effectively prevent merging the main stake account into the pool
    - The funds in the main stake account would not be accessible.
      They can't be moved anywhere by the staker, and also `WithdrawStake` wouldn't be able to withdraw from them because it disallows withdrawing from a non-active validator.

Exploit Scenario #2:

- Bob hijacks a transient account
- Bob delegates to a different validator and makes it usable by the pool
- Now any attempt to increase or decrease the stake would fail, since we'd attempt to merge into an account with a different validator.
  This only way to change the stake in the DoS-ed validator would be to remove it and add it again, and this would need to be done every time an attacker executes this attack.

Recommendations: Don't let hijacked accounts to enter the system in the first place, this can be done by ignoring the transient stake account if we don't expect it to have any funds (if `transient_stake_lamports` is zero).

Customer's response: Fixed in PR #4, hijacked accounts aren't accounted for anymore. Also the update-valiadator-balance function now ensures deactivation of transient stake for validators marked for removal..

Fix Review: Fixed.

H-02 A hijacked account can be used to steal funds from the pool, by removing and re-adding the validator

Severity: **High**	Impact: **High**	Likelihood: **Medium**
Files: program/src/processor.rs	Status: Fixed	Violated Properties: status_update_validator_list_balance status_update_validator_list_balance_no_merge

Description: The pool accounts for a hijacked account as long as it's usable by the pool. However, it accounts for that even when the validator's status is `ReadyForRemoval`. Allowing us to remove funds from the pool after they've been donated.
An attacker with access to the staker role can exploit this to steal funds from the pool.

Exploit Scenario:

- Pool has 1K SOL in its balance, and 1e12 shares
- Bob decreases funds from the pool, and hijacks the accounts once it's merged into the reserve
  - Bob sets the seed to the default value (0)
- Bob removes the validator from the pool and waits for it to be under the `ReadyForRemoval` status
- Bob deposits 1K SOL into the pool
  - Current balance is 2K SOL and 2e12 shares
- Bob donates 2000 SOL to the transient account
  - Pool's balance: 4K SOL, 2e12 shares
- Bob withdraws 1e12-1 shares in exchange for 2K SOL - 2 lamports
  - Bob withdraws that from other sources than the hijacked transient account
  - Pool's balance: 2K SOL + 2 lamports, 1e12+1 shares
- Bob calls cleanup to remove the validator
  - Pool's balance: 2 lamports, 1e12+1 shares
- Bob deposits 1 SOL

    - Pool's balance: 1 SOL + 2 lamports, ~5e20 shares
    - Bob now has the vast majority of the shares in the pool, over 99.99%
- Bob re-adds the validator using the staker role
    - Pool's balance: ~ 2001 SOL, ~5e20 shares
- Bob withdraws his shares
    - Bob would get almost the full 2001 SOL
    - The pool was effectively drained by Bob

Recommendations:

1. Don't allow removing a validator that has a non-zero balance (in the internal accounting)
2. Avoid accounting for hijacked accounts

Customer's response: Fixed in PR #4, both recommendations were implemented

Fix Review: Fixed.

H-03 `WithdrawStake` and removal after cluster-restart don't reset preferred validators after their removal, DoS-ing withdrawals and deposits

Severity: **High**	Impact: **High**	Likelihood: **Medium**
Files: program/src/processor.rs	Status: Fixed	

Description: `WithdrawStake` allows removing validators once there's no active or transient stake left.
However, when it's being removed, we don't check if it's a preferred validator.
This can lead to a state where the preferred validator isn't in the list anymore, causing any attempt to call `WithdrawStake` (once there's active or transient stake in the pool again, this can be done by donating a small amount to one of the stake accounts) or to `DepositStake` to revert.

Same thing happens when a validator is removed after a cluster restart.

Recommendations: Reset the preferred validators' fields if they're being removed during stake withdrawal or cluster restart.

Customer's response: Fixed in PR #4, added a reset to `WithdrawStake`. As for cluster-restart - we've added a check in `CleanupRemovedValidatorEntries` that resets the preferred validators if they're not in the list or not active.

Fix Review: Fixed.

H-04. WithdrawStake allows draining (and hijacking) the reserve stake account, leading to loss of funds.

Severity: **High**	Impact: **High**	Likelihood: **Medium**
Files: program/src/processor.rs	Status: Fixed	Violated Properties: reserve_usable_withdraw_stake_from_reserve

Description: Function `process_withdraw_stake` allows withdrawing from the reserve stake account, without checking that the reserve stake will retain a minimum balance.

This would make the reserve go into Uninitialized state. This basically allows hijacking the reserve stake account.

Once the reserve is hijacked, the attacker can use it to manipulate the share price (or even mint free shares by depositing to the reserve which the attacker controls) and steal any funds remaining in the stake accounts. Note that the remaining funds in the stake accounts would be very little, given they should be below the minimum at the time of the exploit. Of course, any funds deposited to the stake accounts after the attack can also be stolen.

Recommendations: Check in `process_withdraw_stake` that the reserve won't be drained completely.

Customer's response: Fixed in PR #4.

Fix Review: Fixed.

Medium Severity Issues

M-01 The pool keeps accounting for a hijackable stake account after cluster restart

Severity: **Medium**	Impact: **High**	Likelihood: **Low**
Files: program/src/processor.rs program/src/state.rs	Status: Fixed	Violated Properties: active_usable_update_validator_list_balance active_usable_update_validator_list_balance_no_merge

Description: After a cluster-restart, the `UpdateValidatorListBalance` instruction merges main stake accounts that are under the `Initialized` stake state, and calls `remove_validator_stake()` to change the status accordingly.
However, `remove_validator_stake()` doesn't modify the status if the current state is `Active`. That means we'll have a hijackable account that the pool accounts for without even checking if it's usable by the pool.
This can be used by an attacker to steal funds from the pool by manipulating the share price - buying shares when the price is low and selling at the inflated price.

```rust
/// Downgrade the status towards ready for removal by removing the validator
/// stake
pub fn remove_validator_stake(&mut self) -> Result<(), ProgramError> {
    let status = StakeStatus::try_from(*self)?;
    let new_self = match status {
        StakeStatus::Active
        | StakeStatus::DeactivatingTransient
        | StakeStatus::ReadyForRemoval => status,
        StakeStatus::DeactivatingAll => StakeStatus::DeactivatingTransient,
        StakeStatus::DeactivatingValidator => StakeStatus::ReadyForRemoval,
    };
    *self = new_self.into();
```

```
    Ok(())
}
```

Exploit Scenario:

- Bob runs a bot that waits for a cluster restart and hijacks main stake accounts when it happens
- A cluster restart happens and the bot hijacks an account successfully
- Bob uses the hijacked account to manipulate the share price up and down, and uses that to steal funds from the pool
    - They can use the `Withdraw` instruction to withdraw the funds from the account to bring the price back down
    - Even with only 1% of the funds in the pool, they can double their initial investment within less than 100 iterations

Recommendations: When merging the main stake account, if the status is still `Active` then change to either `DeactivatingValidator` or `ReadyForRemoval`, depending on the current state of the transient stake account.

Customer's response: Fixed in PR#4 - we now update the status to the right status (`ReadyForRemoval` or `DeactivatingTransient`), and also always check that the stake is usable by the pool.

Fix Review: Fixed.

M-02 `WithdrawStake` can remove a validator with transient stake, allowing manipulation of share price

Severity: **Medium**	Impact: **High**	Likelihood: **Low**
Files: program/src/processor.rs	Status: Partly fixed	

Description: `WithdrawStake` sets the withdraw source to `ValidatorRemoval` if both `has_active_stake` and `has_transient_stake` are false. This would change the validator's status to `ReadyForRemoval` at the end of the instruction, removing it from the validators list the next time the cleanup instruction is called.
However, `has_transient_stake` is false even when there are some funds left in the transient account, as long as they're below `minimum_lamports_with_tolerance`.
This amount is above the minimum required by increase/decrease stake and by the withdraw stake, so it's not difficult to reach this state.

This can be used by an attacker to hide funds from the pool, and lower the share price.
The amount would be quite low per validator, but with enough time, the staker can lower the share price by a significant amount.

Recommendations: If there are *any* funds in the transient account, set the withdraw source to `Transient` rather than `ValidatorRemoval`

Customer's response: Fixed in PR #4 - we now check if there are any lamports in the transient account before setting the withdraw source to `ValidatorRemoval`

Fix Review: Withdrawal of the last lamports in the transient account is still blocked by a check that the remaining lamports in the transient account are above some threshold.

M-03 Non-active validators block `WithdrawStake` despite not allowing withdrawing from them

Severity: **Medium**	Impact: **High**	Likelihood: **Low**
Files: program/src/processor.rs	Status: Fixed	

Description: In `WithdrawStake` we don't allow withdrawing from the reserve as long as there's active or transient stake, and don't allow withdrawing from transient account as long as there's active stake.
However, when checking if there's active or transient or active stake we account also for stake from non-active validators which we don't allow to withdraw from.

This can lead to a permanent DoS attack by having all of the funds in the reserve and continuously having a validator that's marked for removal, blocking the withdrawal from the reserve.

Exploit Scenario:

- Staker marks all validators for removal
- Staker adds many new validators with minimum amount held in them
- Manager blocks withdrawals from the reserve via the sol withdrawal authority
- When we reach the epoch where the marked-for-removal validator is ready for merge - the staker removes one of the validators with the minimum amount, and donates a few lamports to trigger `has_active_stake = true`
- Permanent DoS achieved, users can't withdraw their funds from the pool

Recommendations: Don't account for non-active validators when checking if there are active or transient funds in the pool.

Customer's response: Fixed in PR #4, we now check only for active validators.

Fix Review: Fixed.

M-04. UpdateValidatorListBalances misses accounting for validator stake funds.

Severity: **Medium**	Impact: **High**	Likelihood: **Low**
Files: program/src/processor.rs	Status: Fixed	Violated Properties: total_lamports_update_validator_list_balance

Description: In `process_update_validator_list_balance` with `no_merge = false` and stake status `DeactivatingValidator` or `DeactivatingAll`, the function misses accounting for validator stake funds in the else case of the following snippet:

```rust
StakeStatus::DeactivatingValidator | StakeStatus::DeactivatingAll => {
        if no_merge {
                active_stake_lamports = validator_stake_info.lamports();
        } else if stake_is_usable_by_pool(
                &meta,
                withdraw_authority_info.key,
                &stake_pool.lockup,
                ) && stake_is_inactive_without_history(&stake, clock.epoch)
        {
                // Validator was removed through normal means.
                // Absorb the lamports into the reserve.
                Self::stake_merge(
                        stake_pool_info.key,
                        validator_stake_info.clone(),
                        withdraw_authority_info.clone(),
                        AUTHORITY_WITHDRAW,
                        stake_pool.stake_withdraw_bump_seed,
                        reserve_stake_info.clone(),
                        clock_info.clone(),
                        stake_history_info.clone(),
                )?;
                validator_stake_record.status.remove_validator_stake()?;
        }
}
```

Recommendations: Account for validator stake funds in the else case.

Customer's response: Fixed in 41b24fe.

Fix Review: Fixed.

Low Severity Issues

L-01. Staker can call `DecreaseStake` on funds when the validator's status is `DeactivatingValidator`, leading to unexpected funds in the transient account.

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: program/src/processor.rs	Status: Fixed	Violated Properties: status_decrease_validator_stake status_decrease_additional_validator_stake status_decrease_validator_stake_with_reserve

Description: The `DecreaseStake` instruction doesn't check the status of the validator, allowing calling this instruction under any status.
The staker might call this instruction while the validator's status is `DeactivatingValidator`, leading to funds being in the transient account while the pool doesn't expect it.
Under the current implementation of `UpdateValidatorListBalance` this doesn't lead to any significant impact, but slight changes might allow merging the main stake alone without transient accounts, or might ignore the transient stake under this state.
If those changes ever happen, this would allow the staker to temporarily 'hide' balance from the pool, using that to manipulate the share price and steal funds from the share-holders.

Recommendations: Prohibit calling `DecreaseStake` on a non-active validator.

Customer's response: Fixed in PR #4

Fix Review: Fixed.

L-02 `WithdrawStake` can be DoS-ed with a huge validators list

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: program/src/processor.rs	Status: Fixed	

Description: `WithdrawStake` would exceed the max CUs per tx if the validators list is huge (around 35K validators).
While this isn't likely to happen under innocent circumstances, a malicious admin can create a pool of that size and populate it, freezing users' funds.

Recommendations: Limit the validator list size (e.g. 20K should be more than enough, and it's well below the DoS threshold) - this can be done during initialization, and also in the `AddValidatorToPool` instruction (to prevent pools created before the patch with a huge validators list from populating it).

Customer's response: Fixed in PR #4, set the limit to 20K

Fix Review: Fixed.

L-03. RemoveValidatorFromPool transitions status to DeactivatingValidator even when there could be funds in transient stake account.

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: program/src/processor.rs	Status: Fixed	Violated Properties: status_remove_validator_to_pool

Description: If `RemoveValidatorFromPool` is called after a cluster restart has moved the transient stake account to `Initialized` state, the status will transition from `Active` to `DeactivatingValidator` state. As a result, the funds in the transient stake account are never merged into the reserve, leading to loss of funds.

Recommendations: Change status to `DeactivatingAll` when there are funds in the transient stake account.

Customer's response: Fixed in PR #4

Fix Review: Fixed.

L-04. WithdrawStake and DecreaseValidatorStake mix physical and virtual accounting leading to an inconsistent status.

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: program/src/processor.rs	Status: Not yet fixed	Violated Properties: status_withdraw_stake_from_validator status_decrease_validator_stake

Description: Instructions `WithdrawStake` and `DecreaseValidatorStake` determine how much to withdraw/decrease on the basis of the lamports in the stake account, followed by decreasing the validator record (i.e. virtual accounting) appropriately. However, a mismatch between physical and virtual accounting could result in a state where the validator status is `Active`, but `validator_record.active_stake_lamports` is zero.

Future changes to the protocol could make such unexpected reachable states exploitable!

Recommendations: Determine how much to withdraw/decrease on the basis of virtual accounting.

Customer's response: Will be fixed in long-term public PRs.

Fix Review:

L-05. Intermediate computation in get_lamports_per_pool_token could overflow.

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: program/src/processor.rs	Status: Not yet fixed	Violated Properties: integrity_get_lamports_per_pool_token

Description: Function `get_lamports_per_pool_token` computes ceiling division via floor division as below:

```rust
pub fn get_lamports_per_pool_token(&self) -> Option<u64> {
    self.total_lamports
        .checked_add(self.pool_token_supply)?
        .checked_sub(1)?
        .checked_div(self.pool_token_supply)
}
```

The intermediate addition could overflow and the function returns None, even though the final answer would be u64. A similar trick is used in the `apply` function as well. However no overflow is possible there as the numbers are converted to u128 first.

Recommendations: Use standard library ceiling division in `get_lamports_per_pool_token()` and `apply()`.

Customer's response: Will be fixed in long-term public PRs.

Fix Review:

Informational Issues

I-01. First depositor attack might be possible if users don't use the slippage check

Description: A first depositor attack might happen in the pool, causing a loss of funds for the users.
A first depositor attack is done by depositing a small amount into an empty pool, then donating funds to increase share price. This would cause the next depositor's amount to round down, causing them a loss of funds.
This would be profitable for the attacker

Recommendation: Make sure the users are aware of this and always use the slippage check while depositing

I-02. Use big_vec in process_initialize for consistency.

Description: Function `process_initialize` uses standard library `Vec` for validator list. This is inconsistent with the usage of `big_vec` everywhere else.

Recommendation: Use `big_vec` in `process_initialize`.

Formal Verification

Verification Methodology

We performed verification of the **Solana Stake Pool** protocol using the Certora verification tool which is based on Satisfiability Modulo Theories (SMT).
In short, the Certora verification tool works by compiling formal specifications written in the Certora Verification Language (CVLR) and the implementation source code written in Rust.
More information about Certora's tooling can be found in the Certora Technology Whitepaper.

If a property is verified with this methodology it means the specification in CVLR holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants: Inductive invariants are proved by induction on the structure of a smart contract. We use constructors/initialization functionality as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.
Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective initialization function. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant. Each such case results in one rule.

Verification Plan

The main focus of the FV audit is to verify the accounting of the Solana Stake Pool protocol. This translates to proving "solvency" of the protocol, i.e. the amount of funds the protocol holds is greater than or equal to the shares issued by the protocol. There are other properties related to solvency, namely that the share price should not drop ("no dilution"), the protocol does not block withdrawal, etc. First, we lay out the verification plan for proving solvency of the Solana Stake Pool. Once the verification plan is set, we discuss how other important properties can be proved as part of this plan.

Our approach is to formalize solvency as "a collection of properties that are stable under the protocol as well as interferences". Let us explain this further. By "a property is stable under the protocol", we mean the following: (i) the property holds after the protocol is initialized; and (ii) for every protocol operation, if we assume that the property holds before the operation, then it must hold after the operation as well. In other words, the property is an invariant of the protocol. By "interference", we mean external operations that change the data over which the protocol operates. By "a property is stable under the interferences", we mean that if the property holds before an interference event, then it must hold after the interference event as well.

Following is the list of interferences that we consider in our verification plan:

1. Lamports of any account may increase (airdrops).
2. Delegated amount of any stake account may increase (rewards accumulation).
3. Stake state can switch from Stake to Initialized (cluster restart).
4. Stake with unknown authority can change non-deterministically (hijacking a closed account).
5. Stake can change non-deterministically while remaining usable by the pool (hijacked account controlled by an attacker).
6. Pool tokens may be burned.

Note that, we do not include any interference like slashing that reduces the lamports associated with an account. This is because the protocol can become insolvent in case of slashing, and no guarantees can be provided in such a scenario.

Going forward, we call this collection of properties formalizing solvency as "solvency invariants". The discussion above can be summarized as follows: if we show that the solvency invariants are stable under the protocol as well as interferences, then we can be certain that any arbitrary sequence of protocol operations and interferences will preserve solvency of the protocol.

Solvency Invariants

Let us now discuss the solvency invariants. We introduce the following useful terms below. The terms in blue represent terms of physical accounting (i.e. data from lamports(), or the token mint), while terms in red represent terms of virtual accounting (i.e. tracked by the stake pool internally).

- PoolTokens_p: total pool tokens minted
- PoolTokens_v: `stake_pool.pool_tokens_supply`
- ReserveBal: lamports in `reserve_stake` without the `minimum_reserve`
- ValidatorBal_v: validator balance stored in `validator_list_info` (`validator_stake_record.active_stake_lamports`)
- ValidatorBal_p: `validator_stake_info.lamports()`
- TransientBal_v: transient balance stored in `validator_list_info` (`validator_stake_record.transient_stake_lamports`)
- TransientBal_p: `transient_stake_info.lamports()`
- TotalLamports_v: `stake_pool.total_lamports`
- TotalLamports_p: total funds controlled by the pool. By definition (summing over validators), TotalLamports_p = ReserveBal + ∑ ValidatorBal_p + ∑ TransientBal_p

The solvency of the stake pool can be expressed as,

PoolTokens_p <= TotalLamports_p.

We are finally ready to express the solvency invariants:

Inv 1. "Pool Tokens Invariant" - the virtual accounting for pool tokens must be greater than the physical accounting, i.e.

PoolTokens_p <= PoolTokens_v.

Inv 2. "Virtual Solvency Invariant' - the stake pool is solvent according to its virtual accounting, ie.,

PoolTokens_v <= TotalLamports_v.

Inv 3. "Total Lamports Invariant" - the virtual accounting of total lamports equals the virtual accounting of all funds in the pool, i.e.,

TotalLamports_v <= ReserveBal + ∑ ValidatorBal_v + ∑ TransientBal_v

Inv 4. "Validator Accounting Invariant" - virtual accounting for validator and transient balances is backed up by physical accounting, i.e., for each validator

ValidatorBal_v <= ValidatorBal_p && TransientBal_v <= TransientBal_p

Combining Inv 1-4, we obtain:

- PoolTokens_p <= PoolTokens_v
- PoolTokens_v <= TotalLamports_v
- TotalLamports_v <= ReserveBal + ∑ ValidatorBal_v + ∑ TransientBal_v
- ReserveBal + ∑ ValidatorBal_v + ∑ TransientBal_v <=
  ReserveBal + ∑ ValidatorBal_p + ∑ TransientBal_p == TotalLamports_p

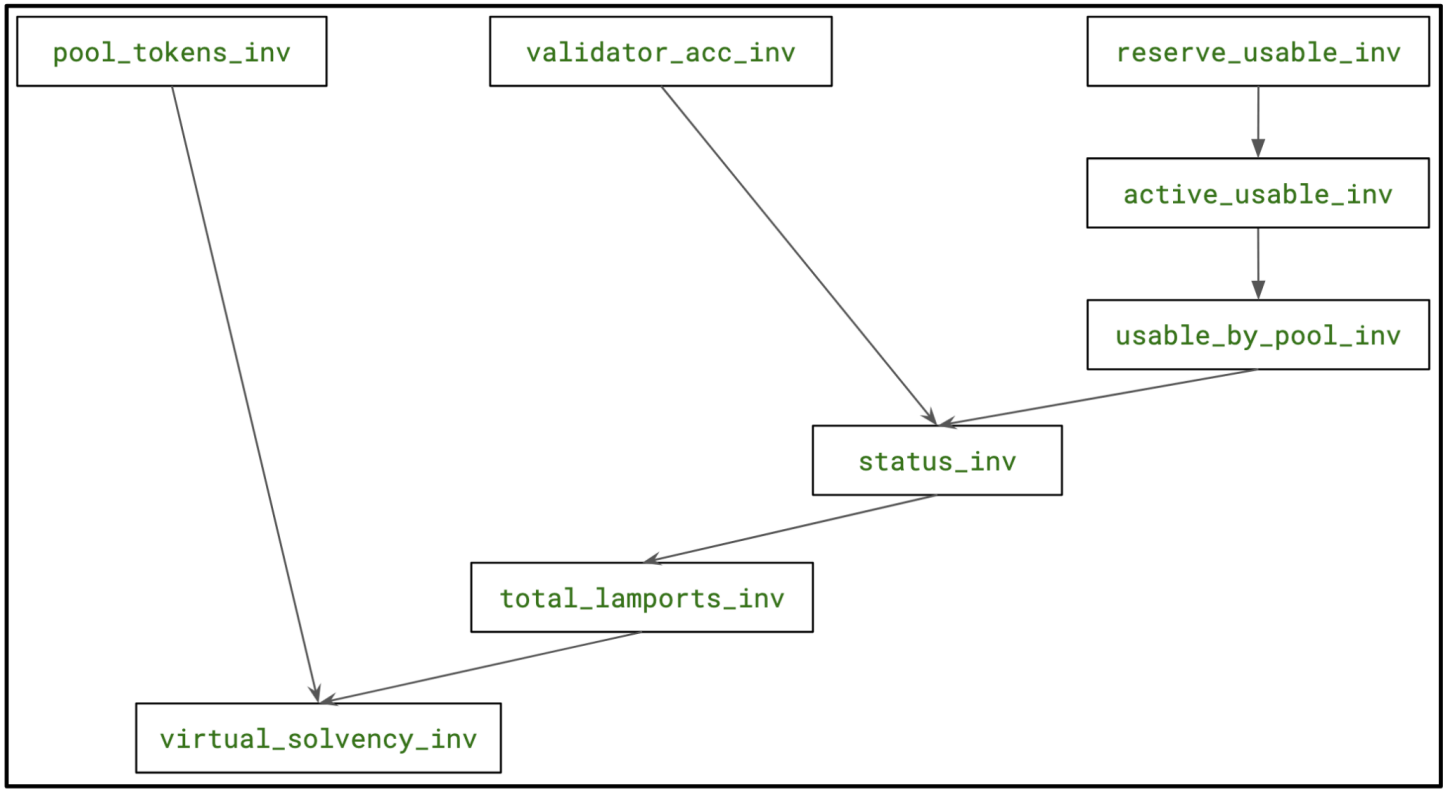
Hence, the solvency invariants indeed show: PoolTokens_p <= TotalLamports_p.

In order to show that Inv 1-4 are stable under the protocol, we require four additional invariants:

- Inv 5: "Reserve Usable Invariant" - the reserve stake is always usable by the pool, i.e.,
  `stake_is_usable_by_pool(reserve_stake_info)`
- Inv 6: "Active Usable Invariant" - any validator with Active status is usable by the pool, i.e.,
  `status` == Active ==> `stake_is_usable_by_pool(validator_stake_info)`
- Inv 7: "Usable by Pool Invariant" - if virtual accounting for any validator stake account (delegated or transient) is greater than zero, then it must be usable by the pool, i.e.,
  ValidatorBal_v > 0 ==> `stake_is_usable_by_pool(validator_stake_info)`
  TransientBal_v > 0 ==> `stake_is_usable_by_pool(transient_stake_info)`
- Inv 8: "Status Invariant" - the validator's status must match the virtual accounting appropriately, i.e.,

```
match status {
    Active ==> ValidatorBal_v > 0,
    DeactivatingTransient ==> ValidatorBal_v == 0 && TransientBal_v > 0,
    ReadyForRemoval ==> ValidatorBal_v == 0 && TransientBal_v == 0,
    DeactivatingValidator ==> ValidatorBal_v > 0 && TransientBal_v == 0,
    DeactivatingAll ==> ValidatorBal_v > 0 && TransientBal_v > 0,
}
```

The interdependence between the invariants is as shown below (Inv A → Inv B means Inv B relies on Inv A):



List of Properties

From the preceding discussion, we distill the following list of properties:

P-1a. “Pool Tokens Invariant” is stable under the protocol.
P-1b. “Pool Tokens Invariant” is stable under interference.
P-2a. “Virtual Solvency Invariant” is stable under the protocol.
P-2b. “Virtual Solvency Invariant” is stable under interference.
P-3a. “Total Lamports Invariant” is stable under the protocol.
P-3b. “Total Lamports Invariant” is stable under interference.
P-4a. “Validator Accounting Invariant” is stable under the protocol.
P-4b. “Validator Accounting Invariant” is stable under interference.
P-5a. “Reserve Usable Invariant” is stable under the protocol.
P-5b. “Reserve Usable Invariant” is stable under interference.
P-6a. “Active Usable Invariant” is stable under the protocol.
P-6b. “Active Usable Invariant” is stable under interference.

P-7a. “Usable by Pool Invariant” is stable under the protocol.
P-7b. “Usable by Pool Invariant” is stable under interference.
P-8a. “Status Invariant” is stable under the protocol.
P-8b. “Status Invariant” is stable under interference.

Additional Properties

- **No Dilution Invariant**: Using the same setup as for proving solvency, we show the property “no dilution of share prices” is stable under the protocol as well as interference. The share price is calculated from the virtual accounting, i.e. PoolTokens_v and TotalLamports_v.
  P-9a. “No Dilution Invariant” is stable under the protocol.
  P-9b. “No Dilution Invariant” is stable under the interference.

- **Epochs are updated correctly**: Critical operations of the protocol must revert if the stake pool accounting has not been updated in the current epoch. We check this by making sure that if the operation succeeds, then the `stake_pool.last_updated_epoch` must be updated to the current epoch. The critical operations include: deposit (sol and stake), withdraw (sol and stake), increase and decrease of validator stake, add and remove validator, and setting fees (when fee can only change at the next epoch).
  P-10. Critical Pool Operations only succeed if the stake pool accounting has been updated.

- **Correctness of Fee Computation**: Fee computation functions respect the monotonicity and bounds as expected.
  P-11. Fee computation functions respect monotonicity and appropriate bounds.

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue
Violated	A counter-example exists that violates one of the assertions of the rule.

General Assumptions and Simplifications

1. Configuration:
    a. Solana Platform Tools - v1.43.
    b. Loops are unrolled at most 2 times.
2. Munging:
    a. `update_validator_list_balance`: reverts when deserialization of a validator stake state fails. This prevents the prover from producing spurious counter-examples.
3. Mocks:
    a. `big_vec` : the data structure has been mocked so as to handle at most two validators.
4. Summarization:
    a. Fee functions are substituted to perform the same arithmetic as before, but via Certora Prover NativeInts Library instead of u64/u128.
    b. We use a simplified version of the Stake Program to make verification tractable for the Certora Prover.
5. `process_initialize` is checked manually to ensure that solvency invariants are established at the end.

Formal Verification Properties Overview

ID	Title	Impact	Status
P-1a, P-1b	Pool Tokens Invariant is stable under the protocol and interference.	Ensures virtual accounting for pool tokens is always greater than or equal to physical tokens.	Verified
P-2a, P-2b	Virtual Solvency Invariant is stable under the protocol and interference.	Ensures solvency of the pool by its virtual accounting.	Verified
P-3b	Total Lamports Invariant is stable under the interference.	Ensures total funds recorded by the pool is the sum of reserve plus validator stake accounts.	Verified
P-4a, P-4b	Validator Accounting Invariant is stable under the protocol and interference.	Ensures that validator virtual accounting is backed up by physical funds.	Verified
P-5b	Reserve Usable Invariant is stable under the interference.	Ensures that the reserve stake is always usable by the pool.	Verified
P-6b	Active Usable Invariant is stable under the interference.	Ensures that any validator stake with Active Status is usable by the pool.	Verified
P-7b	Usable by Pool Invariant is stable under the interference.	Ensures that any validator stake with non-zero recorded funds is usable by the pool.	Verified
P-8b	Status Invariant is stable under the interference.	Ensures that the validator status is consistent with the recorded funds.	Verified
P-9a, P-9b	No Dilution Invariant is stable under the protocol and interference.	Ensures there is no dilution of share prices.	Verified

P-10	Critical Pool Operations only succeed if the stake pool accounting has been updated.	Ensures that the stake pool has been updated before critical protocol operations.	Verified
P-3a	Total Lamports Invariant is stable under the protocol.	Catches bug described in Issue M-02.	Violated
P-5a	Reserve Usable Invariant is stable under the protocol.	Catches bug described in Issue H-04.	Violated
P-6a	Active Usable Invariant is stable under the protocol.	Catches bug described in Issue M-01.	Violated
P-7a	Usable by Pool Invariant is stable under the protocol.	Catches bug described in Issue H-01.	Violated
P-8a	Status Invariant is stable under the protocol.	Catches bug described in Issue H-02, L-01, L-03 and L-04.	Violated
P-11	Correctness of Fee Computation	Catches bug described in Issue L-05.	Violated

Detailed Properties

Module: Pool Tokens Invariant

Module Properties

P-1a. Pool Tokens Invariant is stable under the protocol.

Status: Verified	**Impact: Ensures virtual accounting for pool tokens is always greater than or equal to physical tokens.**

Rule Name	Status	Description	Link to rule report
pool_tokens_add_validator_to_pool	Verified	*Inductive rule for process_add_validator_to_pool.*	*Summarized Report*
pool_tokens_remove_validator_to_pool	Verified	*Inductive rule for process_remove_validator_from_pool.*	
pool_tokens_cleanup_removed_validator_entries	Verified	*Inductive rule for process_cleanup_removed_validator_entries.*	
pool_tokens_set_preferred_validator	Verified	*Inductive rule for process_set_preferred_validator.*	
pool_tokens_set_fee	Verified	*Inductive rule for process_set_fee.*	
pool_tokens_set_manager	Verified	*Inductive rule for process_set_manager.*	
pool_tokens_set_staker	Verified	*Inductive rule for process_set_staker.*	
pool_tokens_set_funding_authority	Verified	*Inductive rule for process_set_funding_authority*	
pool_tokens_deposit_sol	Verified	*Inductive rule for process_deposit_sol.*	

pool_tokens_deposit_stake	Verified	*Inductive rule for process_deposit_stake.*
pool_tokens_withdraw_sol	Verified	*Inductive rule for process_withdraw_stake.*
pool_tokens_withdraw_sol_slippage	Verified	*Inductive rule for process_withdraw_sol with slippage protection.*
pool_tokens_withdraw_stake_from_validator	Verified	*Inductive rule for process_withdraw_stake; case withdrawing from reserve stake.*
pool_tokens_withdraw_stake_from_transient	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a validator_stake.*
pool_tokens_withdraw_stake_from_reserve	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a transient_stake.*
pool_tokens_decrease_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is false and no ephemeral stake account is passed.*
pool_tokens_decrease_additional_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and an ephemeral stake account is passed.*
pool_tokens_decrease_validator_stake_with_reserve	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and no ephemeral stake account is passed.*
pool_tokens_increase_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case no ephemeral stake account is passed.*

pool_tokens_increase_additional_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case an ephemeral stake account is passed.*	
pool_tokens_update_stake_pool_balance	Verified	*Inductive rule for process_update_stake_pool_balance.*	
pool_tokens_update_validator_list_balance	Verified	*Inductive rule for process_update_validator_list_balance; case no_merge is false.*	
pool_tokens_update_validator_list_balance_no_merge	Verified	*Inductive rule for process_update_validator_list_balance; case no_merge is true.*	

P-1b. Pool Tokens Invariant is stable under interference.

Status: Verified	**Impact: Ensures virtual accounting for pool tokens is always greater than or equal to physical tokens.**

Rule Name	Status	Description	Link to rule report
pool_tokens_lamports_may_increase	Verified	*Invariant is stable under the interference lamports of any account may increase.*	*Summarized Report*
pool_tokens_stake_delegation_may_increase	Verified	*Invariant is stable under the interference stake delegation may increase.*	
pool_tokens_stake_may_transition_to_initialized	Verified	*Invariant is stable under the interference that a stake may transition to Initialized state from Stake state.*	

pool_tokens_stake_with_unknown_authority_nondet	Verified	*Invariant is stable under the interference a stake with unknown authority can change nondeterministically.*	
pool_tokens_stake_usable_by_pool_nondet	Verified	*Invariant is stable under the interference that can change nondeterministically while remaining usable by the pool.*	
pool_tokens_pool_tokens_may_be_burned	Verified	*Invariant is stable under the interference that the pool tokens may be burned.*	

Module: Virtual Solvency Invariant

Module Properties

P-2a. Virtual Solvency Invariant is stable under the protocol.

Status: Verified	**Impact: Ensures solvency of the pool by its virtual accounting.**

Rule Name	Status	Description	Link to rule report
virtual_solvency_add_validator_to_pool	Verified	*Inductive rule for process_add_validator_to_pool.*	Summarized Report Summarized Report
virtual_solvency_remove_validator_to_pool	Verified	*Inductive rule for process_remove_validator_from_pool.*	
virtual_solvency_cleanup_removed_validator_entries	Verified	*Inductive rule for process_cleanup_removed_validator_entries.*	
virtual_solvency_set_preferred_validator	Verified	*Inductive rule for process_set_preferred_validator.*	
virtual_solvency_set_fee	Verified	*Inductive rule for process_set_fee.*	
virtual_solvency_set_manager	Verified	*Inductive rule for process_set_manager.*	
virtual_solvency_set_staker	Verified	*Inductive rule for process_set_staker.*	
virtual_solvency_set_funding_authority	Verified	*Inductive rule for process_set_funding_authority*	
virtual_solvency_deposit_sol	Verified	*Inductive rule for process_deposit_sol.*	

virtual_solvency_deposit_stake	Verified	*Inductive rule for process_deposit_stake.*
virtual_solvency_withdraw_sol	Verified	*Inductive rule for process_withdraw_stake.*
virtual_solvency_withdraw_sol_slippage	Verified	*Inductive rule for process_withdraw_sol with slippage protection.*
virtual_solvency_withdraw_stake_from_validator	Verified	*Inductive rule for process_withdraw_stake; case withdrawing from reserve stake.*
virtual_solvency_withdraw_stake_from_transient	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a validator_stake.*
virtual_solvency_withdraw_stake_from_reserve	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a transient_stake.*
virtual_solvency_decrease_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is false and no ephemeral stake account is passed.*
virtual_solvency_decrease_additional_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and an ephemeral stake account is passed.*
virtual_solvency_decrease_validator_stake_with_reserve	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and no ephemeral stake account is passed.*
virtual_solvency_increase_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case no ephemeral stake account is passed.*

virtual_solvency_increase_additional_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case an ephemeral stake account is passed.*	
virtual_solvency_update_stake_pool_balance	Verified	*Inductive rule for process_update_stake_pool_balance.*	
virtual_solvency_update_validator_list_balance	Verified	*Inductive rule for process_update_validator_list_balance; case no_merge is false.*	
virtual_solvency_update_validator_list_balance_no_merge	Verified	*Inductive rule for process_update_validator_list_balance; case no_merge is true.*	

P-2b. Virtual Solvency Invariant is stable under interference.

Status: Verified	**Impact: Ensures solvency of the pool by its virtual accounting.**

Rule Name	Status	Description	Link to rule report
virtual_solvency_lamports_may_increase	Verified	*Invariant is stable under the interference lamports of any account may increase.*	*Summarized Report*
virtual_solvency_stake_delegation_may_increase	Verified	*Invariant is stable under the interference stake delegation may increase.*	
virtual_solvency_stake_may_transition_to_initialized	Verified	*Invariant is stable under the interference that a stake may transition to Initialized state from Stake state.*	

virtual_solvency_stake_with_unknown_authority_nondet	Verified	*Invariant is stable under the interference a stake with unknown authority can change nondeterministically.*	
virtual_solvency_stake_usable_by_pool_nondet	Verified	*Invariant is stable under the interference that can change nondeterministically while remaining usable by the pool.*	
virtual_solvency_pool_tokens_may_be_burned	Verified	*Invariant is stable under the interference that the pool tokens may be burned.*	

Module: Total Lamports Invariant

Module Properties

P-3a. Total Lamports Invariant is stable under the protocol.

Status: Violated	**Impact: Ensures total funds recorded by the pool is the sum of reserve plus validator stake accounts. Catches bug described in Issue M-04.**

Rule Name	Status	Description	Link to rule report
total_lamports_add_validator_to_pool	Verified	*Inductive rule for process_add_validator_to_pool.*	Summarized Report Summarized Report
total_lamports_remove_validator_to_pool	Verified	*Inductive rule for process_remove_validator_from_pool.*	
total_lamports_cleanup_removed_validator_entries	Verified	*Inductive rule for process_cleanup_removed_validator_entries.*	
total_lamports_set_preferred_validator	Verified	*Inductive rule for process_set_preferred_validator.*	
total_lamports_set_fee	Verified	*Inductive rule for process_set_fee.*	
total_lamports_set_manager	Verified	*Inductive rule for process_set_manager.*	
total_lamports_set_staker	Verified	*Inductive rule for process_set_staker.*	
total_lamports_set_funding_authority	Verified	*Inductive rule for process_set_funding_authority*	
total_lamports_deposit_sol	Verified	*Inductive rule for process_deposit_sol.*	

total_lamports_deposit_stake	Verified	*Inductive rule for process_deposit_stake.*
total_lamports_withdraw_sol	Verified	*Inductive rule for process_withdraw_stake.*
total_lamports_withdraw_sol_slippage	Verified	*Inductive rule for process_withdraw_sol with slippage protection.*
total_lamports_withdraw_stake_from_validator	Verified	*Inductive rule for process_withdraw_stake; case withdrawing from reserve stake.*
total_lamports_withdraw_stake_from_transient	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a validator_stake.*
total_lamports_withdraw_stake_from_reserve	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a transient_stake.*
total_lamports_decrease_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is false and no ephemeral stake account is passed.*
total_lamports_decrease_additional_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and an ephemeral stake account is passed.*
total_lamports_decrease_validator_stake_with_reserve	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and no ephemeral stake account is passed.*
total_lamports_increase_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case no ephemeral stake account is passed.*

total_lamports_increase_additional_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case an ephemeral stake account is passed.*	
total_lamports_update_stake_pool_balance	Verified	*Inductive rule for process_update_stake_pool_balance.*	
total_lamports_update_validator_list_balance	Violated	*Inductive rule for process_update_validator_list_balance; case no_merge is false. Violated due to M-04.*	
total_lamports_update_validator_list_balance_no_merge	Verified	*Inductive rule for process_update_validator_list_balance; case no_merge is true.*	

P-3b. Total Lamports Invariant is stable under interference.

Status: Verified	**Impact: Ensures total funds recorded by the pool is the sum of reserve plus validator stake accounts.**

Rule Name	Status	Description	Link to rule report
total_lamports_lamports_may_increase	Verified	*Invariant is stable under the interference lamports of any account may increase.*	Summarized Report
total_lamports_stake_delegation_may_increase	Verified	*Invariant is stable under the interference stake delegation may increase.*	
total_lamports_stake_may_transition_to_initialized	Verified	*Invariant is stable under the interference that a stake may transition to Initialized state from Stake state.*	

total_lamports_stake_with_unknown_authority_nondet	Verified	*Invariant is stable under the interference a stake with unknown authority can change nondeterministically.*	
total_lamports_stake_usable_by_pool_nondet	Verified	*Invariant is stable under the interference that can change nondeterministically while remaining usable by the pool.*	
total_lamports_pool_tokens_may_be_burned	Verified	*Invariant is stable under the interference that the pool tokens may be burned.*	

Module: Validator Accounting Invariant

Module Properties

P-4a. Validator Accounting Invariant is stable under the protocol.

Status: Verified	**Impact: Ensures that validator virtual accounting is backed up by physical funds.**

Rule Name	Status	Description	Link to rule report
validator_accounting_add_validator_to_pool	Verified	*Inductive rule for process_add_validator_to_pool.*	*Summarized Report*
validator_accounting_remove_validator_to_pool	Verified	*Inductive rule for process_remove_validator_from_pool.*	
validator_accounting_cleanup_removed_validator_entries	Verified	*Inductive rule for process_cleanup_removed_validator_entries.*	
validator_accounting_set_preferred_validator	Verified	*Inductive rule for process_set_preferred_validator.*	
validator_accounting_set_fee	Verified	*Inductive rule for process_set_fee.*	
validator_accounting_set_manager	Verified	*Inductive rule for process_set_manager.*	
validator_accounting_set_staker	Verified	*Inductive rule for process_set_staker.*	
validator_accounting_set_funding_authority	Verified	*Inductive rule for process_set_funding_authority*	
validator_accounting_deposit_sol	Verified	*Inductive rule for process_deposit_sol.*	

validator_accounting_deposit_stake	Verified	*Inductive rule for process_deposit_stake.*
validator_accounting_withdraw_sol	Verified	*Inductive rule for process_withdraw_stake.*
validator_accounting_withdraw_sol_slippage	Verified	*Inductive rule for process_withdraw_sol with slippage protection.*
validator_accounting_withdraw_stake_from_validator	Verified	*Inductive rule for process_withdraw_stake; case withdrawing from reserve stake.*
validator_accounting_withdraw_stake_from_transient	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a validator_stake.*
validator_accounting_withdraw_stake_from_reserve	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a transient_stake.*
validator_accounting_decrease_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is false and no ephemeral stake account is passed.*
validator_accounting_decrease_additional_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and an ephemeral stake account is passed.*
validator_accounting_decrease_validator_stake_with_reserve	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and no ephemeral stake account is passed.*
validator_accounting_increase_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case no ephemeral stake account is passed.*

validator_accounting_increase_additional_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case an ephemeral stake account is passed.*	
validator_accounting_update_stake_pool_balance	Verified	*Inductive rule for process_update_stake_pool_balance.*	
validator_accounting_update_validator_list_balance	Verified	*Inductive rule for process_update_validator_list_balance; case no_merge is false.*	
validator_accounting_update_validator_list_balance_no_merge	Verified	*Inductive rule for process_update_validator_list_balance; case no_merge is true.*	

P-4b. Validator Accounting Invariant is stable under interference.

Status: Verified	**Impact: Ensures that validator virtual accounting is backed up by physical funds.**

Rule Name	Status	Description	Link to rule report
validator_accounting_lamports_may_increase	Verified	*Invariant is stable under the interference lamports of any account may increase.*	*Summarized Report*
validator_accounting_stake_delegation_may_increase	Verified	*Invariant is stable under the interference stake delegation may increase.*	
validator_accounting_stake_may_transition_to_initialized	Verified	*Invariant is stable under the interference that a stake may transition to Initialized state from Stake state.*	

validator_accounting_stake_with_unknown_authority_nondet	Verified	*Invariant is stable under the interference a stake with unknown authority can change nondeterministically.*	
validator_accounting_stake_usable_by_pool_nondet	Verified	*Invariant is stable under the interference that can change nondeterministically while remaining usable by the pool.*	
validator_accounting_pool_tokens_may_be_burned	Verified	*Invariant is stable under the interference that the pool tokens may be burned.*	

Module: Reserve Usable Invariant

Module Properties

P-5a. Reserve Usable Invariant is stable under the protocol.

Status: Violated	**Impact: Ensures that the reserve stake is always usable by the pool. Catches bug described in Issue H-04.**

Rule Name	Status	Description	Link to rule report
reserve_usable_add_validator_to_pool	Verified	*Inductive rule for process_add_validator_to_pool.*	*Summarized Report*
reserve_usable_remove_validator_to_pool	Verified	*Inductive rule for process_remove_validator_from_pool.*	
reserve_usable_cleanup_removed_validator_entries	Verified	*Inductive rule for process_cleanup_removed_validator_entries.*	
reserve_usable_set_preferred_validator	Verified	*Inductive rule for process_set_preferred_validator.*	
reserve_usable_set_fee	Verified	*Inductive rule for process_set_fee.*	
reserve_usable_set_manager	Verified	*Inductive rule for process_set_manager.*	
reserve_usable_set_staker	Verified	*Inductive rule for process_set_staker.*	
reserve_usable_set_funding_authority	Verified	*Inductive rule for process_set_funding_authority*	
reserve_usable_deposit_sol	Verified	*Inductive rule for process_deposit_sol.*	

reserve_usable_deposit_stake	Verified	*Inductive rule for process_deposit_stake.*
reserve_usable_withdraw_sol	Verified	*Inductive rule for process_withdraw_stake.*
reserve_usable_withdraw_sol_slippage	Verified	*Inductive rule for process_withdraw_sol with slippage protection.*
reserve_usable_withdraw_stake_from_validator	Verified	*Inductive rule for process_withdraw_stake; case withdrawing from reserve stake.*
reserve_usable_withdraw_stake_from_transient	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a validator_stake.*
reserve_usable_withdraw_stake_from_reserve	Violated	*Inductive rule for process_withdraw_stake; case withdraw from a transient_stake. Violated due to H-04.*
reserve_usable_decrease_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is false and no ephemeral stake account is passed.*
reserve_usable_decrease_additional_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and an ephemeral stake account is passed.*
reserve_usable_decrease_validator_stake_with_reserve	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and no ephemeral stake account is passed.*
reserve_usable_increase_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case no ephemeral stake account is passed.*

reserve_usable_increase_additional_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case an ephemeral stake account is passed.*	
reserve_usable_update_stake_pool_balance	Verified	*Inductive rule for process_update_stake_pool_balance.*	
reserve_usable_update_validator_list_balance	Verified	*Inductive rule for process_update_validator_list_balance; case no_merge is false.*	
reserve_usable_update_validator_list_balance_no_merge	Verified	*Inductive rule for process_update_validator_list_balance; case no_merge is true.*	

P-5b. Reserve Usable Invariant is stable under interference.

Status: Verified	**Impact: Ensures that the reserve stake is always usable by the pool.**

Rule Name	Status	Description	Link to rule report
reserve_usable_lamports_may_increase	Verified	*Invariant is stable under the interference lamports of any account may increase.*	Summarized Report
reserve_usable_stake_delegation_may_increase	Verified	*Invariant is stable under the interference stake delegation may increase.*	
reserve_usable_stake_may_transition_to_initialized	Verified	*Invariant is stable under the interference that a stake may transition to Initialized state from Stake state.*	

reserve_usable_stake_with_unknown_authority_nondet	Verified	*Invariant is stable under the interference a stake with unknown authority can change nondeterministically.*	
reserve_usable_stake_usable_by_pool_nondet	Verified	*Invariant is stable under the interference that can change nondeterministically while remaining usable by the pool.*	
reserve_usable_pool_tokens_may_be_burned	Verified	*Invariant is stable under the interference that the pool tokens may be burned.*	

Module: Active Usable Invariant

Module Properties

P-6a. Active Usable Invariant is stable under the protocol.

Status: Violated	**Impact: Ensures that any validator stake with Active Status is usable by the pool. Catches bug described in Issue M-01.**

Rule Name	Status	Description	Link to rule report
active_usable_add_validator_to_pool	Verified	*Inductive rule for process_add_validator_to_pool.*	Summarized Report Summarized Report
active_usable_remove_validator_to_pool	Verified	*Inductive rule for process_remove_validator_from_pool.*	
active_usable_cleanup_removed_validator_entries	Verified	*Inductive rule for process_cleanup_removed_validator_entries.*	
active_usable_set_preferred_validator	Verified	*Inductive rule for process_set_preferred_validator.*	
active_usable_set_fee	Verified	*Inductive rule for process_set_fee.*	
active_usable_set_manager	Verified	*Inductive rule for process_set_manager.*	
active_usable_set_staker	Verified	*Inductive rule for process_set_staker.*	
active_usable_set_funding_authority	Verified	*Inductive rule for process_set_funding_authority*	
active_usable_deposit_sol	Verified	*Inductive rule for process_deposit_sol.*	

active_usable_deposit_stake	Verified	*Inductive rule for process_deposit_stake.*
active_usable_withdraw_sol	Verified	*Inductive rule for process_withdraw_stake.*
active_usable_withdraw_sol_slippage	Verified	*Inductive rule for process_withdraw_sol with slippage protection.*
active_usable_withdraw_stake_from_validator	Verified	*Inductive rule for process_withdraw_stake; case withdrawing from reserve stake.*
active_usable_withdraw_stake_from_transient	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a validator_stake.*
active_usable_withdraw_stake_from_reserve	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a transient_stake.*
active_usable_decrease_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is false and no ephemeral stake account is passed.*
active_usable_decrease_additional_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and an ephemeral stake account is passed.*
active_usable_decrease_validator_stake_with_reserve	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and no ephemeral stake account is passed.*
active_usable_increase_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case no ephemeral stake account is passed.*

active_usable_increase_additional_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case an ephemeral stake account is passed.*	
active_usable_update_stake_pool_balance	Verified	*Inductive rule for process_update_stake_pool_balance.*	
active_usable_update_validator_list_balance	Violated	*Inductive rule for process_update_validator_list_balance; case no_merge is false. Violated due to M-01.*	
active_usable_update_validator_list_balance_no_merge	Violated	*Inductive rule for process_update_validator_list_balance; case no_merge is true. Violated due to M-01.*	

P-6b. Active Usable Invariant is stable under interference.

Status: Verified	**Impact: Ensures that any validator stake with Active Status is usable by the pool.**

Rule Name	Status	Description	Link to rule report
active_usable_lamports_may_increase	Verified	*Invariant is stable under the interference lamports of any account may increase.*	Summarized Report
active_usable_stake_delegation_may_increase	Verified	*Invariant is stable under the interference stake delegation may increase.*	
active_usable_stake_may_transition_to_initialized	Verified	*Invariant is stable under the interference that a stake may transition to Initialized state from Stake state.*	

active_usable_stake_with_unknown_authority_nondet	Verified	*Invariant is stable under the interference a stake with unknown authority can change nondeterministically.*	
active_usable_stake_usable_by_pool_nondet	Verified	*Invariant is stable under the interference that can change nondeterministically while remaining usable by the pool.*	
active_usable_pool_tokens_may_be_burned	Verified	*Invariant is stable under the interference that the pool tokens may be burned.*	

Module: Usable by Pool Invariant

Module Properties

P-7a. Usable by Pool Invariant is stable under the protocol.

Status: Violated	**Impact: Ensures that any validator stake with non-zero recorded funds is usable by the pool. Catches bug described in Issue H-01.**

Rule Name	Status	Description	Link to rule report
stake_usable_add_validator_to_pool	Verified	*Inductive rule for process_add_validator_to_pool.*	Summarized Report Summarized Report
stake_usable_remove_validator_to_pool	Verified	*Inductive rule for process_remove_validator_from_pool.*	
stake_usable_cleanup_removed_validator_entries	Verified	*Inductive rule for process_cleanup_removed_validator_entries.*	
stake_usable_set_preferred_validator	Verified	*Inductive rule for process_set_preferred_validator.*	
stake_usable_set_fee	Verified	*Inductive rule for process_set_fee.*	
stake_usable_set_manager	Verified	*Inductive rule for process_set_manager.*	
stake_usable_set_staker	Verified	*Inductive rule for process_set_staker.*	
stake_usable_set_funding_authority	Verified	*Inductive rule for process_set_funding_authority*	
stake_usable_deposit_sol	Verified	*Inductive rule for process_deposit_sol.*	

stake_usable_deposit_stake	Verified	*Inductive rule for process_deposit_stake.*
stake_usable_withdraw_sol	Verified	*Inductive rule for process_withdraw_stake.*
stake_usable_withdraw_sol_slippage	Verified	*Inductive rule for process_withdraw_sol with slippage protection.*
stake_usable_withdraw_stake_from_validator	Verified	*Inductive rule for process_withdraw_stake; case withdrawing from reserve stake.*
stake_usable_withdraw_stake_from_transient	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a validator_stake.*
stake_usable_withdraw_stake_from_reserve	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a transient_stake.*
stake_usable_decrease_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is false and no ephemeral stake account is passed.*
stake_usable_decrease_additional_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and an ephemeral stake account is passed.*
stake_usable_decrease_validator_stake_with_reserve	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and no ephemeral stake account is passed.*
stake_usable_increase_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case no ephemeral stake account is passed.*

stake_usable_increase_additional_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case an ephemeral stake account is passed.*	
stake_usable_update_stake_pool_balance	Verified	*Inductive rule for process_update_stake_pool_balance.*	
stake_usable_update_validator_list_balance	Verified	*Inductive rule for process_update_validator_list_balance; case no_merge is false.*	
stake_usable_update_validator_list_balance_no_merge	Violated	*Inductive rule for process_update_validator_list_balance; case no_merge is true. Violated due to H-01.*	

P-7b. Usable by Pool Invariant is stable under interference.

Status: Verified	**Impact: Ensures that any validator stake with non-zero recorded funds is usable by the pool.** **Catches bug described in Issue H-01.**

Rule Name	Status	Description	Link to rule report
usable_by_pool_lamports_may_increase	Verified	*Invariant is stable under the interference lamports of any account may increase.*	Summarized Report
usable_by_pool_stake_delegation_may_increase	Verified	*Invariant is stable under the interference stake delegation may increase.*	
usable_by_pool_stake_may_transition_to_initialized	Verified	*Invariant is stable under the interference that a stake may transition to Initialized state from Stake state.*	

usable_by_pool_stake_with_unknown_authority_nondet	Verified	*Invariant is stable under the interference a stake with unknown authority can change nondeterministically.*	
usable_by_pool_stake_usable_by_pool_nondet	Verified	*Invariant is stable under the interference that can change nondeterministically while remaining usable by the pool.*	
usable_by_pool_pool_tokens_usable_by_poolmay_be_burned	Verified	*Invariant is stable under the interference that the pool tokens may be burned.*	

Module: Status Invariant

Module Properties

P-8a. Status Invariant is stable under the protocol.

Status: Violated	**Impact: Ensures that the validator status is consistent with the recorded funds.** **Catches bug described in Issue H-02, L-01, L-03 and L-04.**

Rule Name	Status	Description	Link to rule report
status_add_validator_to_pool	Verified	*Inductive rule for process_add_validator_to_pool.*	Summarized Report Summarized Report
status_remove_validator_to_pool	Violated	*Inductive rule for process_remove_validator_from_pool. Violated due to L-03.*	
status_cleanup_removed_validator_entries	Verified	*Inductive rule for process_cleanup_removed_validator_entries.*	
status_set_preferred_validator	Verified	*Inductive rule for process_set_preferred_validator.*	
status_set_fee	Verified	*Inductive rule for process_set_fee.*	
status_set_manager	Verified	*Inductive rule for process_set_manager.*	
status_set_staker	Verified	*Inductive rule for process_set_staker.*	
status_set_funding_authority	Verified	*Inductive rule for process_set_funding_authority*	
status_deposit_sol	Verified	*Inductive rule for process_deposit_sol.*	

status_deposit_stake	Verified	*Inductive rule for process_deposit_stake.*
status_withdraw_sol	Verified	*Inductive rule for process_withdraw_stake.*
status_withdraw_sol_slippage	Verified	*Inductive rule for process_withdraw_sol with slippage protection.*
status_withdraw_stake_from_validator	Violated	*Inductive rule for process_withdraw_stake; case withdrawing from reserve stake. Violated due to L-04.*
status_withdraw_stake_from_transient	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a validator_stake.*
status_withdraw_stake_from_reserve	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a transient_stake.*
status_decrease_validator_stake	Violated	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is false and no ephemeral stake account is passed. Violation due to L-01 and L-04.*
status_decrease_additional_validator_stake	Violated	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and an ephemeral stake account is passed. Violation due to L-01.*
status_decrease_validator_stake_with_reserve	Violated	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and no ephemeral stake account is passed. Violation due to L-01.*
status_increase_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case no ephemeral stake account is passed.*

status_increase_additional_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case an ephemeral stake account is passed.*	
status_update_stake_pool_balance	Verified	*Inductive rule for process_update_stake_pool_balance.*	
status_update_validator_list_balance	Violated	*Inductive rule for process_update_validator_list_balance; case no_merge is false. Violation due to H-02.*	
status_update_validator_list_balance_no_merge	Violated	*Inductive rule for process_update_validator_list_balance; case no_merge is true. Violation due to H-02.*	

P-8b. Status Invariant is stable under interference.

Status: Verified	**Impact: Ensures that the validator status is consistent with the recorded funds.**

Rule Name	Status	Description	Link to rule report
status_lamports_may_increase	Verified	*Invariant is stable under the interference lamports of any account may increase.*	Summarized Report
status_stake_delegation_may_increase	Verified	*Invariant is stable under the interference stake delegation may increase.*	
status_stake_may_transition_to_initialized	Verified	*Invariant is stable under the interference that a stake may transition to Initialized state from Stake state.*	
status_stake_with_unknown_a	Verified	*Invariant is stable under the interference a stake with unknown authority can change nondeterministically.*	

uthority_nondet			
status_stake_usable_by_pool_nondet	Verified	*Invariant is stable under the interference that can change nondeterministically while remaining usable by the pool.*	
status_pool_tokens_may_be_burned	Verified	*Invariant is stable under the interference that the pool tokens may be burned.*	

Module: No Dilution Invariant

Module Properties

P-9a. No Dilution Invariant is stable under the protocol.

Status: Verified	**Impact: Ensures there is no dilution of share prices.**

Rule Name	Status	Description	Link to rule report
no_dilution_add_validator_to_pool	Verified	*Inductive rule for process_add_validator_to_pool.*	Summarized Report Summarized Report Summarized Report
no_dilution_remove_validator_to_pool	Verified	*Inductive rule for process_remove_validator_from_pool.*	
no_dilution_cleanup_removed_validator_entries	Verified	*Inductive rule for process_cleanup_removed_validator_entries.*	
no_dilution_set_preferred_validator	Verified	*Inductive rule for process_set_preferred_validator.*	
no_dilution_set_fee	Verified	*Inductive rule for process_set_fee.*	
no_dilution_set_manager	Verified	*Inductive rule for process_set_manager.*	
no_dilution_set_staker	Verified	*Inductive rule for process_set_staker.*	
no_dilution_set_funding_authority	Verified	*Inductive rule for process_set_funding_authority*	
no_dilution_deposit_sol	Verified	*Inductive rule for process_deposit_sol.*	
no_dilution_deposit_stake	Verified	*Inductive rule for process_deposit_stake.*	

no_dilution_withdraw_sol	Verified	*Inductive rule for process_withdraw_stake.*
no_dilution_withdraw_sol_slippage	Verified	*Inductive rule for process_withdraw_sol with slippage protection.*
no_dilution_withdraw_stake_from_validator	Verified	*Inductive rule for process_withdraw_stake; case withdrawing from reserve stake.*
no_dilution_withdraw_stake_from_transient	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a validator_stake.*
no_dilution_withdraw_stake_from_reserve	Verified	*Inductive rule for process_withdraw_stake; case withdraw from a transient_stake.*
no_dilution_decrease_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is false and no ephemeral stake account is passed.*
no_dilution_decrease_additional_validator_stake	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and an ephemeral stake account is passed.*
no_dilution_decrease_validator_stake_with_reserve	Verified	*Inductive rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and no ephemeral stake account is passed.*
no_dilution_increase_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case no ephemeral stake account is passed.*
no_dilution_increase_additional_validator_stake	Verified	*Inductive rule for process_increase_validator_stake; case an ephemeral stake account is passed.*

no_dilution_update_stake_pool_balance	Verified	*Inductive rule for process_update_stake_pool_balance.*	
no_dilution_update_validator_list_balance	Verified	*Inductive rule for process_update_validator_list_balance; case no_merge is false.*	
no_dilution_update_validator_list_balance_no_merge	Verified	*Inductive rule for process_update_validator_list_balance; case no_merge is true.*	

P-9b. No Dilution Invariant is stable under interference.

Status: Verified	**Impact: Ensures there is no dilution of share prices.**

Rule Name	Status	Description	Link to rule report
no_dilution_lamports_may_increase	Verified	*Invariant is stable under the interference lamports of any account may increase.*	Summarized Report
no_dilution_stake_delegation_may_increase	Verified	*Invariant is stable under the interference stake delegation may increase.*	
no_dilution_stake_may_transition_to_initialized	Verified	*Invariant is stable under the interference that a stake may transition to Initialized state from Stake state.*	
no_dilution_stake_with_unknown_authority_nondet	Verified	*Invariant is stable under the interference a stake with unknown authority can change nondeterministically.*	

no_dilution_stake_usable_by_pool_nondet	Verified	*Invariant is stable under the interference that can change nondeterministically while remaining usable by the pool.*	
no_dilution_pool_tokens_may_be_burned	Verified	*Invariant is stable under the interference that the pool tokens may be burned.*	

Module: Epochs are updated correctly

Module Properties

P-10. Critical Pool Operations only succeed if the stake pool accounting has been updated.

Status: Verified	**Impact: Ensures that the stake pool has been updated before critical protocol operations.**

Rule Name	Status	Description	Link to rule report
rule_updated_add_validator_to_pool	Verified	*Rule for process_add_validator_to_pool.*	*Summarized Report*
rule_updated_remove_validator_from_pool	Verified	*Rule for process_remove_validator_from_pool.*	
rule_updated_deposit_sol	Verified	*Rule for process_deposit_sol.*	
rule_updated_deposit_stake	Verified	*Rule for process_deposit_stake.*	
rule_updated_withdraw_sol	Verified	*Rule for process_withdraw_sol.*	
rule_updated_withdraw_sol_slippage	Verified	*Rule for process_withdraw_sol with slippage protection.*	
rule_updated_withdraw_stake	Verified	*Rule for process_withdraw_stake.*	
rule_updated_increase_validator_stake	Verified	*Rule for process_increase_validator_stake; ; case no ephemeral stake account is passed.*	

rule_updated_increase_additional_validator_stake	Verified	*Rule for process_increase_validator_stake; ; case an ephemeral stake account is passed.*	
rule_updated_decrease_validator_stake	Verified	*Rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is false and no ephemeral stake account is passed.*	
rule_updated_decrease_validator_stake_with_reserve	Verified	*Rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and no ephemeral stake account is passed.*	
rule_updated_decrease_additional_validator_stake	Verified	*Rule for process_decrease_validator_stake; case fund_rent_exempt_from_reserve is true and an ephemeral stake account is passed.*	
rule_updated_set_fee	Verified	*Rule for process_set_fee when fee can only be changed in the next epoch.*	

Module: Correctness of Fee Computation

Module Properties

P-11. Fee computation functions respect the monotonicity and bounds as expected.

Status: Violated	**Impact: Correctness of fee computation. Catches bug described in Issue L-05.**

Rule Name	Status	Description	Link to rule report
bound_calc_lamports_withdraw_amount	Verified	*Check on bounds for function calc_lamports_withdraw_amount.*	*Summarized Report*
bound_calc_pool_tokens_sol_referral_fee	Verified	*Check on bounds for function calc_pool_tokens_sol_referral_fee.*	
bound_calc_pool_tokens_stake_referral_fee	Verified	*Check on bounds for function calc_pool_tokens_stake_referral_fee.*	
bound_fee_apply	Verified	*Check on bounds for function apply.*	
integrity_fee_apply	Verified	*Integrity rule for function apply to check that it computes ceiling mulDiv .*	
integrity_get_lamports_per_pool_token	Violated	*Integrity rule for function get_lamports_per_pool_token to check that it computes ceiling div. Violation due to L-05.*	
monotone_calc_lamports_withdraw_amount	Verified	*Checking monotonicity for function calc_lamports_withdraw_amount.*	

monotone_calc_pool_tokens_sol_referral_fee	Verified	*Checking monotonicity for function calc_pool_tokens_sol_referral_fee.*	
monotone_calc_pool_tokens_stake_referral_fee	Verified	*Checking monotonicity for function calc_pool_tokens_stake_referral_fee.*	
monotone_fee_apply	Verified	*Checking monotonicity for function apply.*	

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.